

VLLM专题（四十）——指标



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#)

确保 v1 版本的 LLM 引擎暴露 v0 版本中所有可用的指标的超集。

1. 目标

实现 v0 和 v1 版本之间的指标一致性。

优先使用场景是通过 Prometheus 访问这些指标，因为我们预计在生产环境中会使用这种方式。

日志支持——即将指标打印到信息日志中——用于更临时的测试、调试、开发和探索性使用场景。

2. 背景

vLLM 中的指标可以分为以下几类：

服务器级指标：这些是全局指标，用于跟踪 LLM 引擎的状态和性能。这些指标通常以 Gauge 或 Counter 的形式暴露在 Prometheus 中。

请求级指标：这些指标用于跟踪单个请求的特征——例如大小和时间。这些指标通常以 Histogram 的形式暴露在 Prometheus 中，通常是 SRE 监控 vLLM 时跟踪的服务水平目标（SLO）。

其核心思想是，“服务器级指标”解释了“请求级指标”为何如此。

2.1 v0 版本的指标

在 v0 版本中，以下指标通过 Prometheus 兼容的 `/metrics` 端点暴露，并使用 `vllm:` 前缀：

服务器级指标

- `vllm:num_requests_running` (Gauge)
- `vllm:num_requests_swapped` (Gauge)
- `vllm:num_requests_waiting` (Gauge)
- `vllm:gpu_cache_usage_perc` (Gauge)
- `vllm:cpu_cache_usage_perc` (Gauge)
- `vllm:gpu_prefix_cache_hit_rate` (Gauge)
- `vllm:cpu_prefix_cache_hit_rate` (Gauge)
- `vllm:cache_config_info` (Gauge)
- `vllm:lora_requests_info` (Gauge)
- `vllm:spec_decode_draft_acceptance_rate` (Gauge)
- `vllm:spec_decode_efficiency` (Gauge)

请求级指标

- `vllm:prompt_tokens_total` (Counter)
- `vllm:generation_tokens_total` (Counter)
- `vllm:request_success_total` (Counter)
- `vllm:spec_decode_num_accepted_tokens_total` (Counter)
- `vllm:spec_decode_num_draft_tokens_total` (Counter)
- `vllm:spec_decode_num_emitted_tokens_total` (Counter)
- `vllm:tokens_total` (Counter)

- `vllm:num_preemptions_total` (Counter)
- `vllm:request_prompt_tokens` (Histogram)
- `vllm:request_generation_tokens` (Histogram)
- `vllm:time_to_first_token_seconds` (Histogram)
- `vllm:time_per_output_token_seconds` (Histogram)
- `vllm:e2e_request_latency_seconds` (Histogram)
- `vllm:request_queue_time_seconds` (Histogram)
- `vllm:request_inference_time_seconds` (Histogram)
- `vllm:request_prefill_time_seconds` (Histogram)
- `vllm:request_decode_time_seconds` (Histogram)
- `vllm:request_max_num_generation_tokens` (Histogram)
- `vllm:iteration_tokens_total` (Histogram)
- `vllm:time_in_queue_requests` (Histogram)
- `vllm:model_forward_time_milliseconds` (Histogram)
- `vllm:model_execute_time_milliseconds` (Histogram)
- `vllm:request_params_n` (Histogram)
- `vllm:request_params_max_tokens` (Histogram)

这些指标在 **Inferencing and Serving -> Production Metrics** 中有详细文档说明。

2.2 Grafana 仪表盘

vLLM 还提供了一个参考示例，展示如何使用 Prometheus 收集和存储这些指标，并通过 Grafana 仪表盘进行可视化。

Grafana 仪表盘中暴露的指标子集为我们指明了哪些指标尤为重要：

- `vllm:e2e_request_latency_seconds_bucket` - 端到端请求延迟（以秒为单位）
- `vllm:prompt_tokens_total` - 每秒处理的 Prompt Tokens
- `vllm:generation_tokens_total` - 每秒生成的 Generation Tokens
- `vllm:time_per_output_token_seconds` - 每个输出 Token 的延迟（TPOT，以秒为单位）
- `vllm:time_to_first_token_seconds` - 首 Token 延迟（TTFT，以秒为单位）
- `vllm:num_requests_running`（以及 `_swapped` 和 `_waiting`）- 处于 RUNNING、WAITING 和 SWAPPED 状态的请求数量
- `vllm:gpu_cache_usage_perc` - vLLM 使用的 GPU 缓存块百分比
- `vllm:request_prompt_tokens` - 请求的 Prompt 长度
- `vllm:request_generation_tokens` - 请求的 Generation 长度
- `vllm:request_success_total` - 完成的请求数量（按完成原因分类：生成 EOS Token 或达到最大序列长度）
- `vllm:request_queue_time_seconds` - 队列时间
- `vllm:request_prefill_time_seconds` - 请求的 Prefill 时间
- `vllm:request_decode_time_seconds` - 请求的 Decode 时间
- `vllm:request_max_num_generation_tokens` - 序列组中的最大生成 Token 数量

关于此仪表盘的设计选择，可以参考相关 PR 中的背景信息。

2.3 Prometheus 客户端库

Prometheus 支持最初是通过 `aioprometheus` 库实现的，但很快切换到了 `prometheus_client`。相关 PR 中讨论了这一决策的理由。

2.4 多进程模式

在 v0 版本中，指标在引擎核心进程中收集，并通过多进程模式在 API 服务器进程中暴露。详见 Pull Request #7279。

2.5 内置 Python/进程指标

以下指标默认由 `prometheus_client` 支持，但在使用多进程模式时不会暴露：

- `python_gc_objects_collected_total`
- `python_gc_objects_uncollectable_total`
- `python_gc_collections_total`
- `python_info`
- `process_virtual_memory_bytes`
- `process_resident_memory_bytes`
- `process_start_time_seconds`
- `process_cpu_seconds_total`
- `process_open_fds`
- `process_max_fds`

如果我们在 v1 版本中不再使用多进程模式，这些指标将重新可用。然而，如果这些指标无法聚合组成 vLLM 实例的所有进程的统计数据，它们的相关性可能存疑。

2.6 v0 版本的 PR 和 Issues

以下是一些与 v0 指标相关的 PR，供参考：

- [Pull Request #1890](#)
- [Pull Request #2316](#)
- [Pull Request #2730](#)
- [Pull Request #4464](#)
- [Pull Request #7279](#)

此外，还可以参考“Even Better Observability”功能，其中详细列出了相关路线图。

3. v1 设计

3.1 v1 相关 PR

以下是与 v1 指标问题（Issue #10582）相关的 v1 PR，供参考：

- Pull Request #11962
- Pull Request #11973
- Pull Request #10907
- Pull Request #12416
- Pull Request #12478
- Pull Request #12516
- Pull Request #12530
- Pull Request #12561
- Pull Request #12579
- Pull Request #12592
- Pull Request #12644

3.2 指标收集

在 v1 版本中，我们希望将计算和开销从引擎核心进程中移出，以最小化每次前向传递之间的时间。

V1 EngineCore 设计的总体思路是：

1. **EngineCore 是内循环**：性能在这里最为关键。
2. **AsyncLLM 是外循环**：这部分与 GPU 执行重叠（理想情况下），因此任何“开销”应尽可能放在这里。因此，`AsyncLLM.output_handler_loop` 是指标记录的理想位置。

我们将通过在前端 API 服务器中收集指标来实现这一点，这些指标基于从引擎核心进程返回到前端的 `EngineCoreOutputs` 中提取的信息。

3.3 间隔计算

我们的许多指标是请求处理过程中各种事件之间的时间间隔。最佳实践是使用基于“单调时间”（`time.monotonic()`）的时间戳，而不是“挂钟时间”（`time.time()`）来计算间隔，因为前者不受系统时钟变化（例如来自 NTP）的影响。

还需要注意的是，单调时钟在不同进程之间是不同的——每个进程都有自己的参考点。因此，比较来自不同进程的单调时间戳是没有意义的。

因此，为了计算间隔，我们必须比较来自同一进程的两个单调时间戳。

3.4 调度器统计

引擎核心进程将从调度器中收集一些关键统计信息——例如，在上一次调度器传递后，已调度或等待的请求数量——并将这些统计信息包含在 `EngineCoreOutputs` 中。

3.5 引擎核心事件

引擎核心还将记录某些每个请求事件的时间戳，以便前端可以计算这些事件之间的间隔。

这些事件包括：

1. **QUEUED** - 当请求被引擎核心接收并添加到调度器队列时。
2. **SCHEDULED** - 当请求首次被调度执行时。
3. **PREEMPTED** - 请求被放回等待队列，以便为其他请求腾出空间。它将在未来重新调度并重新开始其预填充阶段。
4. **NEW_TOKENS** - 当 `EngineCoreOutput` 中包含的输出生成时。由于这是给定迭代中所有请求共有的，我们使用 `EngineCoreOutputs` 上的单个时间戳来记录此事件。

计算的间隔包括：

1. **队列间隔** - 从 QUEUED 到最近的 SCHEDULED 之间的时间。
2. **预填充间隔** - 从最近的 SCHEDULED 到随后的第一个 NEW_TOKENS 之间的时间。
3. **解码间隔** - 从第一个（在最近的 SCHEDULED 之后）到最后一个 NEW_TOKENS 之间的时间。
4. **推理间隔** - 从最近的 SCHEDULED 到最后一个 NEW_TOKENS 之间的时间。
5. **Token 间间隔** - 连续的 NEW_TOKENS 之间的时间。

换句话说：这些间隔帮助我们更好地理解请求在不同阶段的时间消耗，从而优化性能和资源分配。

我们探讨了让前端根据其可见的事件时间来计算这些间隔的可能性。然而，前端无法观察到 **QUEUED** 和 **SCHEDULED** 事件的时间，而由于我们需要基于同一进程的单调时间戳来计算间隔.....因此需要引擎核心记录所有这些事件的时间戳。

3.5.1 间隔计算与抢占

当在解码阶段发生抢占时，由于任何已生成的 Token 会被重用，我们将抢占视为影响以下间隔：

1. **Token 间间隔**
2. **解码间隔**
3. **推理间隔**

换句话说，抢占会延长这些间隔，因为请求需要重新调度并可能重新开始部分处理阶段。

当在预填充过程中发生抢占（假设此类事件是可能的）时，我们认为抢占会影响到首次生成 token 的时间和预填充间隔。

3.6 前端统计收集

当前端处理单个 EngineCoreOutputs（即单次引擎核心迭代的输出）时，它会收集与该迭代相关的各种统计数据：

- 本次迭代中生成的新 token 总数。
- 本次迭代中完成的预填充处理的 prompt token 总数。
- 本次迭代中调度的任何请求的队列间隔。
- 本次迭代中完成预填充的任何请求的预填充间隔。
- 本次迭代中所有请求的 inter-token 间隔（每输出 token 的时间，TPOT）。
- 本次迭代中完成预填充的任何请求的首次 token 时间（TTFT）。然而，我们计算这个时间间隔是相对于请求首次接收到前端（arrival_time）的时间，以便考虑输入处理时间。

对于在给定迭代中完成的任何请求，我们还记录以下信息：

- 推理和解码间隔——相对于调度和首次 token 事件，如上所述。
- 端到端延迟——前端 arrival_time 到前端接收到最后一个 token 之间的间隔。

3.7 指标发布 - 日志记录

LoggingStatLogger 指标发布器每 5 秒输出一一次 INFO 日志消息，包含一些关键指标：

- 当前正在运行/等待的请求数量。
- 当前 GPU 缓存使用情况。
- 过去 5 秒钟内处理的 prompt token 数量。
- 过去 5 秒钟内生成的新 token 数量。
- 最近 1000 次 kv-cache 块查询的前缀缓存命中率。

3.8 指标发布 - Prometheus

PrometheusStatLogger 指标发布器通过 Prometheus 兼容的格式在 /metrics HTTP 端点提供指标。然后可以配置 Prometheus 实例轮询该端点（例如，每秒一次）并将其值记录在时间序列数据库中。

Prometheus 通常通过 Grafana 使用，从而能够随时间绘制这些指标。

Prometheus 支持以下指标类型：

- **计数器（Counter）**：一个会随时间增加的值，永远不会减少，并且通常在 vLLM 实例重启时重置为零。例如，实例生命周期内生成的 token 数量。
- **仪表（Gauge）**：一个会上下波动的值，例如当前调度执行的请求数量。
- **直方图（Histogram）**：记录在不同桶中的指标样本数量。例如，TTFT 小于 1 毫秒、5 毫秒、10 毫秒、20 毫秒等请求的数量。

Prometheus 指标也可以加标签，允许根据匹配的标签组合指标。在 vLLM 中，我们会给每个指标添加一个 model_name 标签，包含该实例提供的模型名称。

示例输出：

```
curl http://0.0.0.0:8000/metrics
# HELP vllm:num_requests_running Number of requests in model execution batches.
# TYPE vllm:num_requests_running gauge
vllm:num_requests_running{
    model_name="meta-llama/Llama-3.1-8B-Instruct"} 8.0
...
# HELP vllm:generation_tokens_total Number of generation tokens processed.
# TYPE vllm:generation_tokens_total counter
vllm:generation_tokens_total{
    model_name="meta-llama/Llama-3.1-8B-Instruct"} 27453.0
...
# HELP vllm:request_success_total Count of successfully processed requests.
# TYPE vllm:request_success_total counter
vllm:request_success_total{
    finished_reason="stop",model_name="meta-llama/Llama-3.1-8B-Instruct"} 1.0
vllm:request_success_total{
    finished_reason="length",model_name="meta-llama/Llama-3.1-8B-Instruct"} 131.0
vllm:request_success_total{
    finished_reason="abort",model_name="meta-llama/Llama-3.1-8B-Instruct"} 0.0
...
# HELP vllm:time_to_first_token_seconds Histogram of time to first token in seconds.
# TYPE vllm:time_to_first_token_seconds histogram
vllm:time_to_first_token_seconds_bucket{
    le="0.001",model_name="meta-llama/Llama-3.1-8B-Instruct"} 0.0
vllm:time_to_first_token_seconds_bucket{
    le="0.005",model_name="meta-llama/Llama-3.1-8B-Instruct"} 0.0
vllm:time_to_first_token_seconds_bucket{
    le="0.01",model_name="meta-llama/Llama-3.1-8B-Instruct"} 0.0
vllm:time_to_first_token_seconds_bucket{
    le="0.02",model_name="meta-llama/Llama-3.1-8B-Instruct"} 13.0
vllm:time_to_first_token_seconds_bucket{
    le="0.04",model_name="meta-llama/Llama-3.1-8B-Instruct"} 97.0
```

```
vllm:time_to_first_token_seconds_bucket{
    le="0.06",model_name="meta-llama/Llama-3.1-8B-Instruct"} 123.0
vllm:time_to_first_token_seconds_bucket{
    le="0.08",model_name="meta-llama/Llama-3.1-8B-Instruct"} 138.0
vllm:time_to_first_token_seconds_bucket{
    le="0.1",model_name="meta-llama/Llama-3.1-8B-Instruct"} 140.0
vllm:time_to_first_token_seconds_count{
    model_name="meta-llama/Llama-3.1-8B-Instruct"} 140.0
```

注意事项

选择适合广泛使用场景的直方图桶并不简单，可能需要随着时间的推移进行调整和优化。

3.9 缓存配置信息

`prometheus_client` 支持 Info 类型的指标，这相当于一个值固定为 1 的仪表（Gauge），但通过标签暴露有趣的键/值对信息。这用于表示某个实例的不可变信息——因此只需要在启动时观察一次，并允许在 Prometheus 中跨实例进行比较。

我们在 `vllm:cache_config_info` 指标中使用了这一概念：

```
# HELP vllm:cache_config_info LLMEngine CacheConfig 信息
# TYPE vllm:cache_config_info gauge
vllm:cache_config_info{
    block_size="16",cache_dtype="auto",calculate_kv_scales="False",cpu_offload_gb
="0",enable_prefix_caching="False",gpu_memory_utilization="0.9",...} 1.0
```

然而，`prometheus_client` 在多进程模式下从未支持 Info 类型的指标，原因不明确。我们简单地使用了一个值为 1 的仪表（Gauge）指标，并将 `multiprocess_mode` 设置为 `"mostrecent"` 来代替。

3.10 LoRA 指标

`vllm:lora_requests_info` 仪表类似于上述指标，不过它的值是当前的墙钟时间，并且会在每次迭代时更新。

使用的标签名包括：

- `running_lora_adapters`：每个适配器正在运行的请求数量，格式为以逗号分隔的字符串。
- `waiting_lora_adapters`：类似地，统计等待调度的请求数量。
- `max_lora`：静态配置，“单批次最多 LoRA 数量”。

用逗号分隔的字符串来编码多个适配器的运行/等待计数似乎不太合适——我们可以使用标签来区分每个适配器的计数，这一点应该重新考虑。

请注意，`multiprocess_mode="livemostrecent"` 被使用——只取当前正在运行进程中的最新指标。

此设计是在 Pull Request #9477 中添加的，且至少已有一个已知用户。如果我们重新考虑此设计并弃用旧的指标，我们应该通过在 v0 中也进行更改并要求该项目迁移到新指标，从而减少长时间的弃用期。

3.11 前缀缓存指标

关于在 Issue #10582 中讨论添加前缀缓存指标，有些有趣的观点可能对我们未来如何处理指标有所启发。

每次查询前缀缓存时，我们记录查询的块数以及缓存中存在的查询块数（即命中数）。

然而，关注的指标是命中率——即每次查询的命中数。

在日志记录的情况下，我们希望用户通过计算最近固定数量的查询的命中率来获得最佳体验（目前该间隔被固定为最近的 1000 次查询）。

而在 Prometheus 的情况下，我们应该利用 Prometheus 的时间序列特性，允许用户计算他们选择的时间间隔内的命中率。例如，使用 PromQL 查询来计算过去 5 分钟的命中间隔：

```
rate(cache_query_hit[5m]) / rate(cache_query_total[5m])
```

为了实现这一点，我们应该将查询和命中数记录为 Prometheus 中的计数器，而不是将命中率记录为仪表（Gauge）。

4. 废弃指标

4.1 如何废弃指标

废弃指标不应轻率处理。用户可能不会注意到某个指标已被废弃，而当该指标突然（从他们的角度来看）被移除时，即使有等效的指标可供使用，也可能会给他们带来很大的不便。

例如，可以参考 `vllm:avg_prompt_throughput_toks_per_s` 是如何被废弃的（代码中有注释）、移除的，以及用户是如何注意到这一变化的。

一般来说：

1. **谨慎废弃指标**：我们应谨慎对待废弃指标，尤其是因为很难预测对用户的影响。
2. 在 `/metrics` 输出中包含显著的废弃通知：在指标的帮组字符串中明确标注废弃信息。
3. 在用户文档和发布说明中列出废弃指标：确保用户能够通过文档了解哪些指标已被废弃。
4. 考虑通过 **CLI** 参数隐藏废弃指标：在完全删除之前，为管理员提供一个过渡期的“逃生舱口”。

4.2 未实现的指标 - `vllm:tokens_total`

该指标由 Pull Request #4464 添加，但显然从未实现。可以直接移除。

4.3 重复的指标 - 队列时间

`vllm:time_in_queue_requests` 是一个 Histogram 指标，由 Pull Request #9659 添加，其计算方式为：

```
self.metrics.first_scheduled_time = now
self.metrics.time_in_queue = now - self.metrics.arrival_time
```

两周后，Pull Request #4464 添加了 `vllm:request_queue_time_seconds`，导致以下代码：

```
if seq_group.is_finished():
    if (seq_group.metrics.first_scheduled_time is not None and
        seq_group.metrics.first_token_time is not None):
        time_queue_requests.append(
            seq_group.metrics.first_scheduled_time -
            seq_group.metrics.arrival_time)
    ...
    if seq_group.metrics.time_in_queue is not None:
        time_in_queue_requests.append(
            seq_group.metrics.time_in_queue)
```

这似乎是重复的，应移除其中一个。由于后者被 Grafana 仪表盘使用，因此应废弃或从 v0 中移除前者。

4.4 前缀缓存命中率

如上所述，我们现在暴露的是“查询次数”和“命中次数”计数器，而不是“命中率”指标。

5. KV缓存卸载

在v0版本中，有两个与“交换”抢占模式相关的指标，在v1版本中该模式已不再适用：

- `vllm:num_requests_swapped`
- `vllm:cpu_cache_usage_perc`

在这种模式下，当一个请求被抢占（例如，为了腾出KV缓存空间以完成其他请求时），我们将KV缓存块交换到CPU内存中。这也被称为“KV缓存卸载”，并通过配置 `--swap-space` 和 `--preemption-mode` 来实现。

在v0版本中，vLLM长期以来支持束搜索。SequenceGroup封装了N个共享相同提示KV块的序列的概念。这使得不同请求之间可以共享KV缓存块，并通过写时复制（copy-on-write）实现分支。CPU交换主要是针对这些类似束搜索的场景。

后来，引入了前缀缓存（prefix caching）的概念，它允许KV缓存块在请求之间隐式共享。与CPU交换相比，这被证明是一个更好的选择，因为缓存块可以按需慢慢逐出，并且被逐出的提示部分可以重新计算。

在v1版本中，SequenceGroup被移除，尽管为了支持“并行采样”（ $n>1$ ），需要一个替代方案。束搜索功能已经被移出核心（在v0版本中）。这是一个非常不常见的功能，且代码非常复杂。

在v1版本中，由于前缀缓存的效率更高（零开销）并且默认启用，抢占和重新计算策略应该会表现得更好。

6. 未来工作

6.1 并行采样

一些v0版本的指标仅在“并行采样”上下文中相关。在并行采样中，请求中的n参数用于从同一个提示词中请求多个生成。

作为在Pull Request #10980中添加并行采样支持的一部分，我们还应该添加以下指标：

vllm:request_params_n (直方图)

观察每个完成请求中的'n'参数值。

vllm:request_max_num_generation_tokens (直方图)

观察每个完成的序列组中所有序列的最大输出长度。如果没有并行采样，这与**vllm:request_generation_tokens**相同。

6.2 推测解码

一些v0版本的指标是特定于“推测解码”的。在推测解码中，我们使用更快的近似方法或模型生成候选标记，然后使用更大的模型验证这些标记。

- **vllm:spec_decode_draft_acceptance_rate (仪表盘)**
- **vllm:spec_decode_efficiency (仪表盘)**
- **vllm:spec_decode_num_accepted_tokens_total (计数器)**
- **vllm:spec_decode_num_draft_tokens_total (计数器)**
- **vllm:spec_decode_num_emitted_tokens_total (计数器)**

目前有一个PR（Pull Request #12193）正在审查，旨在将“提示查找（ngram）”推测解码功能添加到v1版本。其他技术也将在之后加入。我们应当在此背景下重新审视v0版本的指标。

注意：我们可能需要像处理前缀缓存命中率那样，将接受率分为已接受和草稿的计数器。同样，效率也可能需要类似的处理。

6.3 自动扩展和负载均衡

我们指标的一个常见使用场景是支持vLLM实例的自动扩展。

关于Kubernetes Serving工作组的相关讨论，请参考以下内容：

- 《Kubernetes中大模型服务器指标的标准化》
- 《在Kubernetes中进行LLM工作负载性能评估与自动扩展基准测试》
- 推理性能
- Issue #5041 和 Pull Request #12726。

这是一个复杂的话题。考虑到Rob的这条评论：

“我认为这个指标应该集中在估算最大并发度，这会导致平均请求长度 > 每秒查询数... 因为这实际上是‘饱和’服务器的原因。”

一个明确的目标是，我们应该暴露出需要的指标，以检测这个饱和点，方便管理员基于这些指标实现自动扩展规则。然而，为了做到这一点，我们需要清晰了解管理员（和自动监控系统）应该如何判断实例接近饱和：

- 如何识别模型服务器计算的饱和点（即无法通过更高的请求率获得更多吞吐量，但开始引入额外的延迟）？这样我们才能有效地进行自动扩展？

6.4 指标命名

我们在命名指标方面的做法可能需要重新审视：

- 在指标名称中使用冒号似乎与“冒号是保留给用户定义的记录规则”这一原则相悖。
- 我们的大多数指标遵循以单位结尾的惯例，但并非所有指标都遵循这一惯例。
- 一些指标名称以 `_total` 结尾：

如果指标名称后缀为 `_total`，它将被移除。在暴露计数器的时间序列时，将添加 `_total` 后缀。这是为了与OpenMetrics和Prometheus文本格式的兼容性，因为OpenMetrics要求添加 `_total` 后缀。

6.5 指标命名

我们在命名指标方面的做法可能需要重新审视：

- 在指标名称中使用冒号似乎与“冒号是保留给用户定义的记录规则”这一原则相悖。
- 我们的大多数指标遵循以单位结尾的惯例，但并非所有指标都遵循这一惯例。
- 一些指标名称以 `_total` 结尾：

如果指标名称后缀为 `_total`，它将被移除。在暴露计数器的时间序列时，将添加 `_total` 后缀。这是为了与OpenMetrics和Prometheus文本格式的兼容性，因为OpenMetrics要求添加 `_total` 后缀。

6.6 添加更多指标

关于新增指标的想法不乏其人：

- 来自其他项目的示例，如TGI。
- 来自特定用例的提议，比如上述提到的Kubernetes自动扩展话题。
- 可能来自标准化努力的提议，例如OpenTelemetry对生成式人工智能（Gen AI）的语义约定。

在添加新指标时，我们应该保持谨慎。虽然添加指标通常相对简单：

- 它们一旦添加，就很难移除——请参见上文关于弃用的部分。
- 启用这些指标时，可能会对性能产生重要影响。通常，除非这些指标能够在生产环境中默认启用，否则它们的实际用途非常有限。
- 它们会影响项目的开发和维护。每添加一个指标，v0版本就会使得v1版本的开发工作变得更加耗时，也许并不是所有的指标都值得为其维护投入如此多的时间和精力。

7. 追踪 - OpenTelemetry

指标提供了系统性能和健康状况随时间的汇总视图，而追踪则跟踪单个请求在不同服务和组件之间的流动。两者都属于更广泛的“可观察性”范畴。

v0版本已支持OpenTelemetry追踪：

- 通过Pull Request #4687添加
- 通过 `--oltp-traces-endpoint` 和 `--collect-detailed-traces` 配置
- OpenTelemetry博客文章
- 用户文档
- 博客文章
- IBM产品文档

OpenTelemetry有一个生成式人工智能工作组（Gen AI Working Group）。

由于指标本身是一个足够大的话题，我们将在v1版本中单独讨论追踪的话题。

7.1 OpenTelemetry模型前向与执行时间

在v0版本中，我们有以下两个指标：

- **vllm:model_forward_time_milliseconds (直方图)** - 请求在批处理期间，模型前向传播所花费的时间。
- **vllm:model_execute_time_milliseconds (直方图)** - 模型执行函数所花费的时间。这包括模型前向传播、跨工作者的块同步/同步、CPU-GPU同步时间和采样时间。

这些指标仅在启用了OpenTelemetry追踪且使用了 `--collect-detailed-traces=all/model/worker` 时才会启用。该选项的文档中说明：

为指定的“模块”收集详细的追踪。这可能涉及使用成本高昂或阻塞操作，因此可能会对性能产生影响。

这些指标是通过Pull Request #7089添加的，并在OpenTelemetry追踪中显示为：

-> gen_ai.latency.time_in_scheduler: Double(0.017550230026245117)

-> gen_ai.latency.time_in_model_forward: Double(3.151565277099609)

-> gen_ai.latency.time_in_model_execute: Double(3.6468167304992676)

我们已经有了 `inference_time` 和 `decode_time` 指标，因此问题在于是否存在足够常见的使用场景，能够证明这些更高分辨率的时间度量是值得的。